

Activity Feed System

Technical Documentation · MERN Stack

1. Project Overview

A tenant-isolated activity feed system built using MongoDB, Express, React, and Node.js. It implements cursor-based pagination, optimistic UI updates, and simulated real-time updates.

2. Key Features

Backend

- Create activity via POST /activities
- Fetch paginated feed via GET /activities
- Tenant isolation using tenantId
- Cursor-based pagination — no offset/skip
- Compound indexing for performance
- Response projection to limit payload size

Frontend

- Infinite-scrolling activity feed
- Optimistic UI updates for instant feedback
- Simulated real-time updates via polling
- Loading and empty-state handling
- Clean, responsive UI

3. Technology Stack

Frontend: React (Hooks)

Backend: Node.js + Express

Database: MongoDB

Styling: CSS

4. API Design

4.1 Create Activity

Endpoint:

```
POST /api/activities
```

Request Body:

```
{
  "actorId": "user_1",
  "actorName": "John Doe",
  "type": "created task",
  "metadata": { "title": "Fix login bug" }
}
```

4.2 Get Activities (Cursor Pagination)

Endpoint:

```
GET /api/activities?cursor=&limit;=10
```

Response:

```
{
  "data": [ /* activity objects */ ],
  "nextCursor": "2026-04-25T10:04:00.000Z"
}
```

5. Cursor-Based Pagination

Instead of offset-based pagination, this system uses a cursor derived from the last item's createdAt timestamp. Each subsequent request fetches only records older than that cursor, enabling O(limit) query performance at any scale.

How it works:

- Fetch latest records sorted by createdAt DESC.
- Store the last item's timestamp as the cursor.
- Next request filters: createdAt < cursor.

```
{ createdAt: { $lt: cursor } }
```

Why not skip()?

- Inefficient for large datasets — scans all skipped records.
- Performance degrades as the offset grows.

Benefits of cursor pagination:

- Uses the index efficiently.
- $O(\text{limit})$ query performance regardless of dataset size.
- Stable — inserts don't cause items to appear twice or be skipped.

6. Performance Optimisation

MongoDB Compound Index:

```
{ tenantId: 1, createdAt: -1 }
```

This index satisfies both the tenant equality filter and the descending time sort in a single scan, eliminating in-memory sorting.

- `tenantId` — equality predicate, applied first for high selectivity.
- `createdAt DESC` — supports range queries and removes the sort stage.
- Covers the cursor filter pattern exactly.

7. Frontend Architecture

- Custom hook (`useActivities`) encapsulates all state and data-fetching logic.
- Strict separation of concerns: API layer → hooks → UI components.
- Infinite scroll implemented with `IntersectionObserver`.
- Controlled re-renders via `useCallback` memoisation.

Optimistic UI Update Flow

1. User submits a new activity.
2. UI renders a temporary item immediately (optimistic update).
3. API request dispatched in the background.
4. On success — replace temporary item with server response.
5. On failure — rollback to previous state and surface an error.

8. Real-Time Updates

Real-time behaviour is simulated using short-interval polling, which is sufficient for prototyping and lightweight deployments.

```
setInterval(fetchActivities, 5000);
```

Production alternatives:

WebSocket: Full-duplex real-time — recommended for production.

SSE: Server-to-client push, suitable for notification streams.

9. Scaling to 50M+ Activities per Tenant

The following strategies keep the system performant as individual tenants accumulate tens of millions of activity records.

9.1 Indexing

- Compound index { tenantId, createdAt } keeps all queries O(limit).
- Partial indexes can further reduce index size for inactive tenants.

9.2 Sharding Strategy

- Shard by tenantId to distribute load horizontally.
- Heavy tenants can be isolated onto dedicated shards.

9.3 Hot Tenant Isolation

- Monitor per-tenant throughput and detect spikes.
- Allocate dedicated replica sets or cloud regions for heavy tenants.

9.4 Data Retention (TTL Index)

- Apply a TTL index to auto-expire old documents and control storage costs.

9.5 Event-Driven Ingestion

- Decouple writes using a queue (BullMQ, Kafka, RabbitMQ).
- Workers persist events asynchronously, lowering API latency.
- Built-in retry and dead-letter handling for fault tolerance.

TTL Index Example:

```
db.activities.createIndex(  
  { createdAt: 1 },  
  { expireAfterSeconds: 60 * 60 * 24 * 90 } // 90 days  
);
```

10. Debugging & Refactoring

Problem: Infinite Re-render Loop

Buggy code:

```
useEffect(() => {  
  fetchActivities();  
}, [activities]); // activities changes on every fetch → infinite loop
```

Including activities in the dependency array causes the effect to re-run on every state update, triggering another fetch, creating an unbounded loop.

Fix:

```
useEffect(() => {  
  fetchActivities();  
}, []); // empty array = run once on mount only
```

Prevention: audit every dependency array and only include values the effect genuinely depends on. Avoid placing mutable state in dependency arrays unless intentional re-execution is required.

11. Key Learnings

- Cursor pagination is essential for scalable, stable infinite feeds.
- Optimistic UI dramatically improves perceived responsiveness.
- Index design is the single biggest lever for MongoDB query performance.
- Separation of concerns (hooks, API layer, UI) improves testability.
- Event-driven architecture decouples ingestion from storage for independent scaling.

12. Conclusion

This project demonstrates a production-oriented approach to building scalable activity feeds. Cursor-based pagination with a well-designed MongoDB index sustains constant query performance as data grows. The React frontend uses optimistic updates and hook-based architecture for a fluid user experience. The design patterns applied — tenant isolation, TTL retention, event-driven ingestion, and hot-tenant isolation — provide a clear path to operating at tens of millions of records per tenant.